

AI Assisted Coding

Assignment 12.3

Name: Nihaal

Hallticket:2303A52498

Task 1: Sorting Student Records for Placement Drive

Scenario

SR University's Training and Placement Cell needs to shortlist candidates efficiently during campus placements. Student records must be sorted by CGPA in descending order.

Tasks

1. Use GitHub Copilot to generate a program that stores student records (Name, Roll Number, CGPA).
2. Implement the following sorting algorithms using AI assistance:

o Quick Sort o

Merge Sort

3. Measure and compare runtime performance for large datasets.
4. Write a function to display the top 10 students based on CGPA.

Expected Outcome

- Correctly sorted student records.
- Performance comparison between Quick Sort and Merge Sort.
- Clear output of top-performing students.

Prompt:

"" assume you are a developer for a university where Placement cell needs to shortlist the students efficiency during campus placement exam must be sorted by CGPA in decending order

you must use the quick sort and merge sort to sort the CGPA of students measure and compare runtime

write a display function to show the top 10 students based on CGPA descending order ""

CODE:

```
> def quick_sort(arr):
>     if len(arr) <= 1:
>         return arr
>     pivot = arr[len(arr) // 2]
>     left = [x for x in arr if x > pivot]
>     middle = [x for x in arr if x == pivot]
>     right = [x for x in arr if x < pivot]
>     return quick_sort(left) + middle + quick_sort(right)
>
> def merge_sort(arr):
>     if len(arr) <= 1:
>         return arr
>     mid = len(arr) // 2
>     left = merge_sort(arr[:mid])
>     right = merge_sort(arr[mid:])
>     return merge(left, right)
>
> def merge(left, right):
>     merged = []
>     i, j = 0, 0
>     while i < len(left) and j < len(right):
>         if left[i] > right[j]:
>             merged.append(left[i])
>             i += 1
>         else:
>             merged.append(right[j])
>             j += 1
>     merged.extend(left[i:])
>     merged.extend(right[j:])
>     return merged
>
> def display_top_students(students, n=10):
>     print(f"Top {n} students based on CGPA:")
>     for i in range(min(n, len(students))):
>         print(f"{i+1}. {students[i][0]} - CGPA: {students[i][1]}")
>
> # Generate random student data
> students = [(f"Student_{i}", round(random.uniform(2.0, 4.0), 2)) for i in range(100)]
> # Sort using quick sort
> start_time = time.time()
> sorted_students_quick = quick_sort(students)
> end_time = time.time()
> print(f"Quick Sort Time: {end_time - start_time:.6f} seconds")
> # Sort using merge sort
> start_time = time.time()
> sorted_students_merge = merge_sort(students)
> end_time = time.time()
```

Output:

```

PS C:\Users\nihaa\AppData\Local\Programs\Microsoft VS Code> &
Quick Sort Time: 0.000197 seconds
Merge Sort Time: 0.000445 seconds

Top 10 students based on CGPA:
1. Student_39 - CGPA: 4.0
2. Student_94 - CGPA: 4.0
3. Student_71 - CGPA: 3.97
4. Student_89 - CGPA: 3.96
5. Student_67 - CGPA: 3.95
6. Student_77 - CGPA: 3.95
7. Student_6 - CGPA: 3.94
8. Student_10 - CGPA: 3.94
9. Student_60 - CGPA: 3.94
10. Student_24 - CGPA: 3.93
PS C:\Users\nihaa\AppData\Local\Programs\Microsoft VS Code>

```

Task_02:

Task 2: Implementing Bubble Sort with AI Comments •

Task: Write a Python implementation of Bubble Sort.

- Instructions:
- Students implement Bubble Sort normally.
- Ask AI to generate inline comments explaining key logic (like swapping, passes, and termination).
- Request AI to provide time complexity analysis.
- Expected Output:
- A Bubble Sort implementation with AI-generated explanatory comments and complexity analysis.

Prompt:

```

""" give me the code for Bubble sort with inline comments and explain the code in detail """
def bubble_sort(arr):
    n = len(arr) # Get the length of the array
    # Traverse through all elements in the array
    for i in range(n):
        # Last i elements are already in place, no need to check them
        for j in range(0, n-i-1):
            # Swap if the element found is greater than the next element
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j] # Swap the elements
    return arr # Return the sorted array
# Example usage
if __name__ == "__main__":
    arr = [64, 34, 25, 12, 22, 11, 90] # Sample array to be sorted
    print("Original array:", arr) # Print the original array
    sorted_arr = bubble_sort(arr) # Sort the array using bubble sort
    print("Sorted array:", sorted_arr) # Print the sorted array
# Explanation of the code:
# The bubble_sort function takes an array as input and sorts it in ascending order using the bubble
# sort algorithm. The outer loop runs n times, where n is the length of the array. The inner loop compares
# adjacent elements and swaps them if they are in the wrong order. After each pass through the array, the largest
# element is "bubbled" to the end of the array, which is why we can ignore the last i elements in subsequent passes.

```

Output:

```

PS C:\Users\nihaa\AppData\Local\Programs\Microsoft VS Code> & c:\python314\python.exe
Original array: [64, 34, 25, 12, 22, 11, 90]
Sorted array: [11, 12, 22, 25, 34, 64, 90]
PS C:\Users\nihaa\AppData\Local\Programs\Microsoft VS Code>

```

Task 3: Quick Sort and Merge Sort Comparison

- Task: Implement Quick Sort and Merge Sort using recursion.

• Instructions:

- Provide AI with partially completed functions for recursion.
- Ask AI to complete the missing logic and add docstrings.
- Compare both algorithms on random, sorted, and reverse-sorted lists.

• Expected Output:

- Working Quick Sort and Merge Sort implementations.

- AI-generated explanation of average, best, and worst-case complexities Prompt:

"" compare the both quick & merge sort using recursion • Compare both algorithms on random, sorted, and reverse-sorted lists. AI-generated explanation of average, best, and worst-case complexities""

```
"" compare the both quick & merge sort using recursion • Compare both algorithms on random, sorted, and reverse-sorted lists. AI-generated explanation of average, best, and worst-case complexities""
import random
import time
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + middle + quick_sort(right)
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)
def merge(left, right):
    merged = []
    i, j = 0, 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1
    merged += left[i:]
    merged += right[j:]
    return merged
```

Output:

```
--- Random List ---
Quick Sort Time: 0.001942 seconds
Merge Sort Time: 0.003646 seconds

--- Sorted List ---
Quick Sort Time: 0.001979 seconds
Merge Sort Time: 0.001479 seconds

--- Reverse Sorted List ---
Quick Sort Time: 0.001395 seconds
Merge Sort Time: 0.001745 seconds
PS C:\Users\nihaa\AppData\Local\Programs\Microsoft VS Code>
```

Task 4 (Real-Time Application – Inventory Management System)

Scenario: A retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to:

1. Quickly search for a product by ID or name.
2. Sort products by price or quantity for stock analysis.

Task:

- Use AI to suggest the most efficient search and sort algorithms for this use case.
- Implement the recommended algorithms in Python.

- Justify the choice based on dataset size, update frequency, and performance requirements.

Expected Output:

- A table mapping operation → recommended algorithm → justification.
- Working Python functions for searching and sorting the inventory.

CODE:

```

171 class Product:
172     def __init__(self, pid, name, price, quantity):
173         self.pid = pid
174         self.name = name
175         self.price = price
176         self.quantity = quantity
177     def __repr__(self):
178         return f"{self.pid} | {self.name} | Price:{self.price} | Qty:{self.quantity}"
179 # Binary Search (Search by Product ID)
180 def binary_search(products, target_id):
181     low = 0
182     high = len(products) - 1
183     while low <= high:
184         mid = (low + high) // 2
185         if products[mid].pid == target_id:
186             return products[mid]
187         elif products[mid].pid < target_id:
188             low = mid + 1
189         else:
190             high = mid - 1
191     return None
192 # Linear Search (Search by Name)
193 def linear_search(products, name):
194     for product in products:
195         if product.name.lower() == name.lower():
196             return product
197     return None
198 # Quick Sort by Price

```

Output:

```

PS C:\Users\nihaa\AppData\Local\Programs\Microsoft VS Code> &
All Products:
101 | Laptop | Price: 800 | Qty: 5
102 | Phone | Price: 500 | Qty: 10
103 | Tablet | Price: 300 | Qty: 7
104 | Headphones | Price: 100 | Qty: 15
105 | Keyboard | Price: 50 | Qty: 20

Binary Search (ID = 103):
103 | Tablet | Price: 300 | Qty: 7

Linear Search (Name = Phone):
102 | Phone | Price: 500 | Qty: 10

Sorted by Price:
105 | Keyboard | Price: 50 | Qty: 20
104 | Headphones | Price: 100 | Qty: 15
103 | Tablet | Price: 300 | Qty: 7
102 | Phone | Price: 500 | Qty: 10
101 | Laptop | Price: 800 | Qty: 5
PS C:\Users\nihaa\AppData\Local\Programs\Microsoft VS Code>

```

Task_05:

Task 5: Real-Time Stock Data Sorting & Searching Scenario:

An AI-powered FinTech Lab at SR University is building a tool for analyzing stock price movements. The requirement is to quickly sort stocks by daily gain/loss and search for specific stock symbols efficiently.

- Use GitHub Copilot to fetch or simulate stock price data (Stock Symbol, Opening Price, Closing Price).
- Implement sorting algorithms to rank stocks by percentage change.
- Implement a search function that retrieves stock data instantly when a stock symbol is entered.
- Optimize sorting with Heap Sort and searching with Hash Maps.
- Compare performance with standard library functions (sorted(), dictlookups) and analyze trade-offs.

Prompt:

""" Generate a short solution for a Python assignment on **Real-Time Stock Data Sorting & Searching**."""

Scenario:

A FinTech lab tool analyzes stock price movements. Each stock contains:

- * Stock Symbol
- * Opening Price
- * Closing Price

Tasks:

1. Simulate a small dataset of stock prices.
2. Calculate percentage change for each stock.
3. Implement **Heap Sort** to rank stocks by percentage gain/loss.
4. Implement **Hash Map (dictionary)** search to quickly retrieve stock data by stock symbol.
5. Compare the performance with Python built-in methods **sorted()** and **dictionary lookups**.
6. Provide:

* Short explanation of the Python functions for sorting and searching *

Example outputs.

"""

Code:

```
import random
# Simulate stock data
stocks = [
    {"symbol": "AAPL", "open": 150, "close": 155},
    {"symbol": "GOOGL", "open": 2800, "close": 2750},
    {"symbol": "AMZN", "open": 3400, "close": 3450},
    {"symbol": "MSFT", "open": 300, "close": 310},
    {"symbol": "TSLA", "open": 700, "close": 680},
]
# Calculate percentage change
for stock in stocks:
    stock["change"] = ((stock["close"] - stock["open"]) / stock["open"]) * 100
# Heap Sort implementation to rank stocks by percentage gain/loss
def heapify(arr, n, i):
    largest = i
    l = 2 * i + 1
    r = 2 * i + 2
    if l < n and arr[l]["change"] > arr[largest]["change"]:
        largest = l
    if r < n and arr[r]["change"] > arr[largest]["change"]:
        largest = r
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)
def heap_sort(arr):
    n = len(arr)
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)
    for i in range(n - 1, 0, -1):
        arr[i], arr[0] = arr[0], arr[i]
        heapify(arr, i, 0)
# Sort stocks by percentage change
heap_sort(stocks)
# Create a hash map (dictionary) for quick retrieval by stock symbol
stock_data = {stock["symbol"]: stock for stock in stocks}
```

Output:

```
PS C:\Users\Vivek\OneDrive\Desktop\3-2\AI-Assisted-Coding\Codes> python -u "c:\User
Stocks sorted by percentage change (Heap Sort):
TSLA: -2.86%
GOOGL: -1.79%
AMZN: 1.47%
MSFT: 3.33%
AAPL: 3.33%

Stocks sorted by percentage change (Built-in sorted()):
MSFT: 3.33%
AMZN: 1.47%
MSFT: 3.33%
AAPL: 3.33%

Stocks sorted by percentage change (Built-in sorted()):
MSFT: 3.33%
Stocks sorted by percentage change (Built-in sorted()):
MSFT: 3.33%
AAPL: 3.33%
AMZN: 1.47%
AAPL: 3.33%
AMZN: 1.47%
AMZN: 1.47%
GOOGL: -1.79%
TSLA: -2.86%
```